

Текст защиты курсовой работы

«Разработка системы беспроводного управления гиростабилизируемой платформой»

Ориентир: 6-8 минут спокойного выступления.

Слайд 1. Титульный слайд

Здравствуйте. Тема моей курсовой работы - разработка системы беспроводного управления гиростабилизируемой платформой. В работе я рассматривал не только отдельный датчик или отдельный интерфейс, а полный программно-аппаратный прототип: микроконтроллер, инерциальный датчик, телеметрию, web-пульт, исполнительный контур и механическую основу корпуса. Основная идея работы - сделать такой стенд, который можно проверять по слоям и дальше использовать для настройки стабилизации.

Слайд 2. Зачем нужна система

Проблема состоит в том, что гиростабилизируемая платформа является мехатронной системой. Она должна измерять положение, оценивать отклонение и формировать управляющее воздействие. В работе рассматривается не абстрактный алгоритм, а связанная инженерная цепочка: датчик передает данные на микроконтроллер, микроконтроллер формирует телеметрию, интерфейс отображает состояние, а команды доходят до привода. Поэтому цель работы - разработать прототип беспроводного управления такой платформой на базе ESP32-P4-M3, MPU-9250, HTTP/WebSocket и 3D-печатного корпуса.

Слайд 3. Что сделано в работе

В результате работы получился программно-аппаратный прототип. Был подключен IMU-датчик MPU-9250 по I2C, подтвержден адрес 0x68 и идентификатор WHO_AM_I 0x71. Также была подготовлена прошивка под ESP32-P4-M3, реализована телеметрия, HTTP API, WebSocket-обмен, web-интерфейс оператора и физическая механическая основа в виде сферического корпуса. При этом я отдельно фиксирую ограничение: финальная доводка коэффициентов стабилизации требует дополнительных механических испытаний уже после полной сборки.

Слайд 4. Слоистая архитектура прототипа

Архитектура сделана слоистой. На нижнем уровне находятся питание, GPIO, I2C, драйвер L293D, ТТ-моторы и корпус. Выше работает embedded-логика: инициализация, чтение MPU, формирование телеметрии и обработка команд. Дальше идет сетевой слой с Wi-Fi, HTTP API, WebSocket и JSON-состоянием. Над ним расположен backend-мост, который нормализует данные, и frontend-пульт оператора. Такое разделение нужно для отладки: если система ведет себя неправильно, можно отдельно проверить питание, отдельно I2C, отдельно датчик, отдельно API, UI и привод.

Слайд 5. Стенд и подключение датчика

Первым критичным результатом была проверка аппаратного подключения. Датчик GY-9250 или MPU-9250 подключался по I2C: SDA на GPIO1, SCL на GPIO2, AD0 на землю для адреса 0x68, а NCS на 3.3 В для режима I2C. Успешным состоянием я считаю два условия одновременно:

сканирование шины видит устройство по адресу 0x68, и чтение регистра WHO_AM_I возвращает 0x71. Без такой проверки нельзя переходить к стабилизации, потому что ошибка может быть не в алгоритме, а в проводах, питании или режиме датчика.

Слайд 6. Web-пульт оператора и телеметрия

Следующий слой - удаленное управление. Интерфейс здесь не декоративный, а диагностический. Через GET /api/telemetry можно получить состояние системы и IMU, через GET /api/wifi - состояние беспроводного интерфейса, через POST /api/command - передать команды Stop, Forward или Reverse, а через WebSocket - получать поток событий. Главное безопасное правило: команда Stop должна быть доступна всегда. Если связь или датчик работают нестабильно, привод должен переводиться в безопасное состояние.

Слайд 7. Алгоритм оценки угла и ПИД-регулятор

Для оценки угла используется сочетание акселерометра и гироскопа. Гироскоп хорошо отражает быстрые изменения, но при интегрировании накапливает дрейф. Акселерометр дает долгосрочную привязку к направлению силы тяжести, но чувствителен к шумам и ускорениям. Поэтому в работе заложен комплементарный фильтр, где α равно 0.98, а период дискретизации - 20 миллисекунд. После оценки угла считается ошибка относительно целевого положения, а управляющее воздействие формируется ПИД-регулятором с начальными коэффициентами K_p 2.0, K_i 0.1 и K_d 0.5.

Слайд 8. Как команда превращается в движение

Важно, что быстрый контур стабилизации не должен замыкаться через браузер или backend. Wi-Fi нужен для включения режима, настройки параметров и просмотра телеметрии, но расчет управляющего воздействия должен оставаться локально на микроконтроллере. Иначе задержки сети будут напрямую влиять на устойчивость системы. Поэтому правильная логика такая: frontend отправляет режим и параметры, embedded-часть выполняет быстрый контур, а обратно отдает угол, ошибку, управляющее воздействие и состояние привода.

Слайд 9. Результаты стендовых испытаний

В испытаниях я проверял не промышленную стабилизацию, а работоспособность ключевой цепочки. Подтверждено чтение MPU-9250, обновление телеметрии около 48.4 Гц, прохождение команд до исполнительного контура и отображение состояния в интерфейсе. В покое измерения показали малую ошибку, но после большого возмущения система переходит в колебательный режим. Это ожидаемый результат для прототипа без финальной механической настройки коэффициентов и центра масс.

Слайд 10. Итоги работы

В завершение отмечу, что в рамках курсовой работы был получен достаточный инженерный результат: IMU, телеметрия, команды, UI и физическая основа корпуса объединены в единый прототип. Дальнейшее развитие связано с более точной механической настройкой, подбором коэффициентов регулятора и расширенными испытаниями, но базовая цель работы выполнена. Такой вывод корректно фиксирует уровень результата: создана проверяемая инженерная основа системы.

Слайд 11. Итоги работы

Итог работы - создан программно-аппаратный прототип системы беспроводного управления гиросtabilизируемой платформой. В нем есть измерение, телеметрия, удаленный интерфейс, команды управления, безопасный режим остановки и физическая механическая база.

Практическая ценность в том, что этот стенд можно использовать для дальнейшей настройки стабилизации, подбора коэффициентов и развития уже более точного мехатронного прототипа.

Спасибо за внимание.

Короткие ответы на возможные вопросы

Почему не Bluetooth?

В текущей архитектуре использован Wi-Fi, потому что он лучше подходит для web-пульта, HTTP API, WebSocket и быстрой диагностики через браузер. Bluetooth можно рассматривать как отдельный транспорт в будущей версии, но он не был основным подтвержденным контуром этой работы.

Почему результат не называется промышленной стабилизацией?

Потому что промышленный результат требует полной механической сборки, точной настройки центра масс, коэффициентов регулятора, PWM-управления и серии повторяемых испытаний. В курсовой подтверждена инженерная основа: датчик, телеметрия, команды, интерфейс и механический прототип.

Зачем нужен web-интерфейс?

Он нужен не для красоты, а для диагностики. Через него можно видеть состояние IMU, связь, режим привода, отправлять безопасные команды и понимать, где именно возникает ошибка.

Почему быстрый контур должен быть на микроконтроллере?

Потому что Wi-Fi и backend не гарантируют стабильную малую задержку. Если стабилизация будет зависеть от браузера, система станет непредсказуемой. Поэтому браузер управляет режимом, а расчет воздействия остается на embedded-части.